Software Engineering & Project Management (SEPM)

Energy & Utility Management System (EUMS)

Billing, metering and customer service for electricity, water and gas

System Architecture

Software process model: Waterfall · Combined SEPM + DBMS course project

Group number	Group __
Course and section	Software Engineering & Project Management (SEPM), Section __
Date	October 2026

#	Name	Registration number
1	Sumit P Surendran	20243590
2	Sreethan Reddy	20243550
3	Sushrut Lad	20243551
4	Viswajith	20243559

1. System Architecture

1.1 Architecture diagram

EUMS is a three-layer client–server app, loosely following MVC: the templates are the view, the Flask route functions are the controller, and the SQLite tables plus `billing.py` are the model. The browser sends normal form GET/POST requests and gets finished HTML pages back. The only JSON endpoint is the one that gives the reading form the last meter reading.

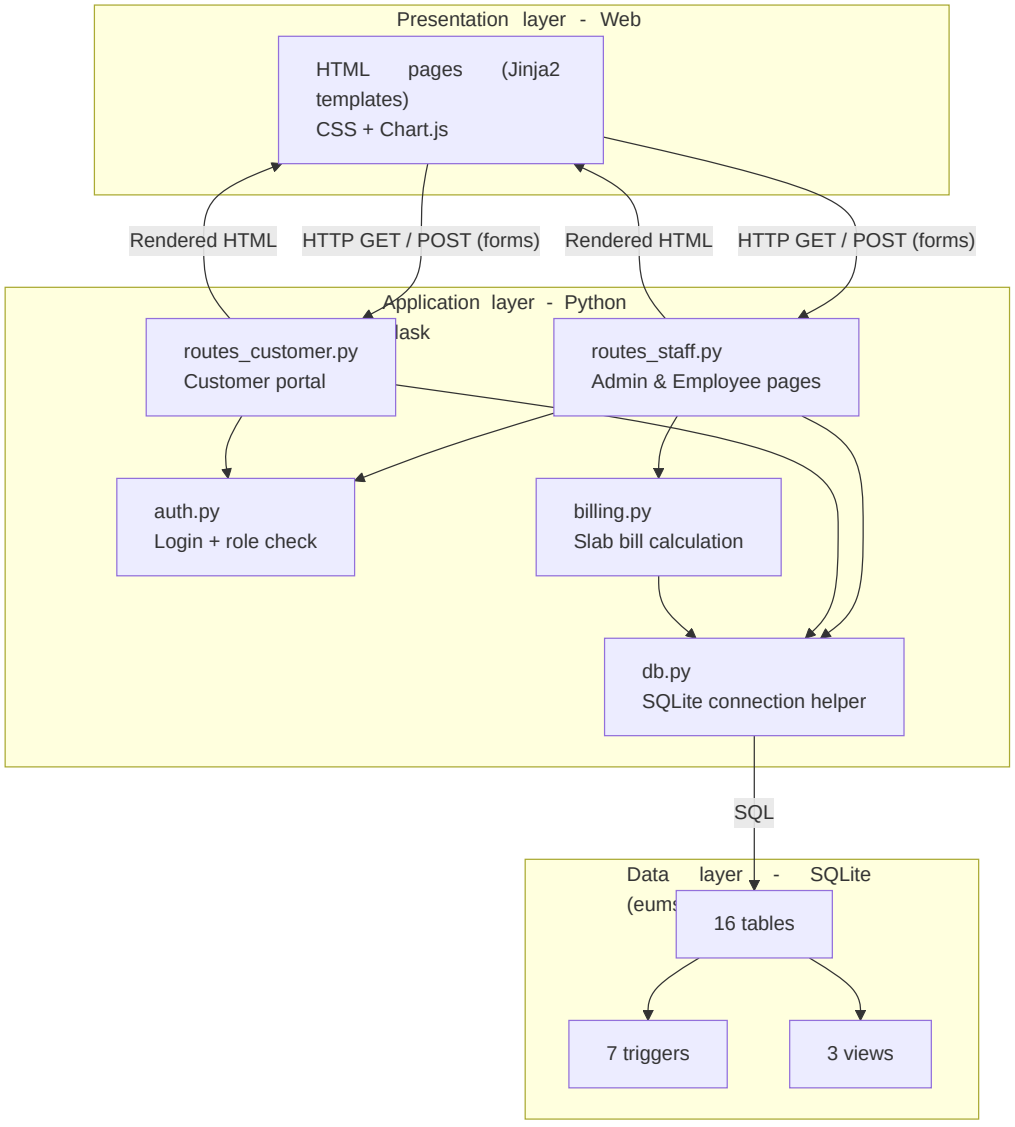


Figure 1: Three-tier architecture of EUMS

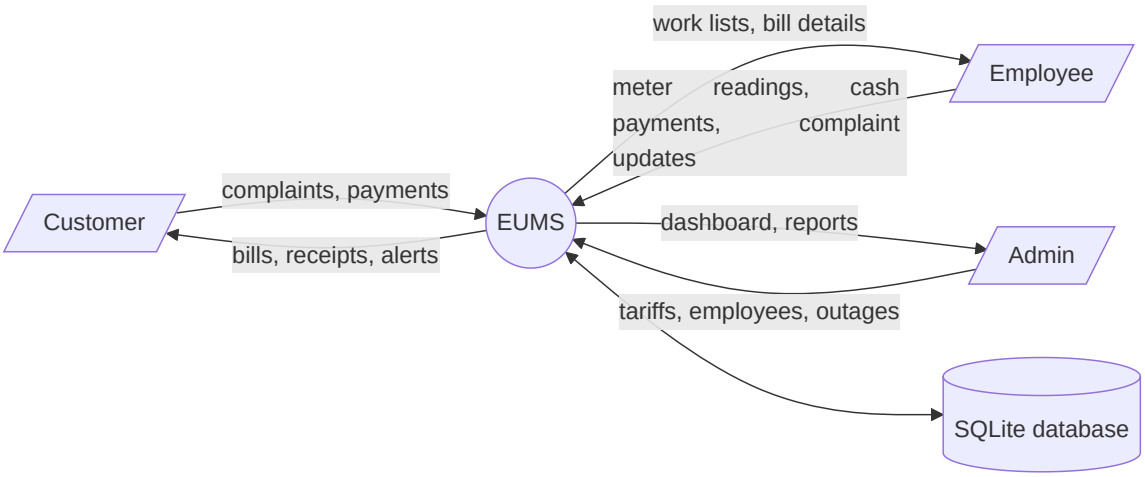


Figure 2: Context-level (Level 0) data flow diagram

1.2 Module description

File / module	Responsibility
`app.py`	Creates the Flask app, loads configuration, registers the two route groups, login/logout pages, error pages, and template helpers (money, date formatting).
`auth.py`	`login_required(*roles)` decorator: redirects to login if not signed in, returns 403 if the role is not allowed.
`db.py`	Opens one SQLite connection per request (foreign keys on, rows as dictionaries) and closes it afterwards; `query()` helper.
`billing.py`	Pure function `calculate_bill()` for slab pricing (unit tested) and `generate_bill()`, which reads the tariff and inserts the bill and bill items.
`routes_staff.py`	Admin and employee pages: dashboard, customers, connections, meter readings, billing, payments, complaints, outages, tariffs, employees, reports.
`routes_customer.py`	Customer portal: dashboard, connections, bills, pay, usage history, complaints, outages, profile, notifications. Every query is filtered by the logged-in customer.
`schema.sql`	All tables, constraints, indexes, triggers and views.
`seed.py`	Creates the database and loads demo data (24 customers from `sample_data/customers.csv`, 6 months of readings, bills, payments, complaints, outages).
`templates/`, `static/`	Jinja2 HTML pages (one per feature), one CSS file, and a small Chart.js helper.
`tests/`	pytest unit tests (billing) and functional tests (whole app with a fresh database per test).

Table 1: Modules